ASSINGMENT 10 – Code Review and Quality:

Using AI to Improve Code

Quality and Readability

Task Description #1 – Syntax and Logic Errors

Prompt: write python code to identify and fix syntax and logic errors in a faulty

Python script.

Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)

correct code:

```
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average
```

marks = [85, 90, 78, 92]

print("Average Score is", calc_average(marks))

Explanation:

The program calculates the total marks using a for loop. It divides the total marks by the number of marks to find the average. Fixed the indentation error, corrected average to average, and added the missing closing parenthesis.

Output:

```
PS C:\Users\prath\.vscode> python -u "c:\Users\prath\.vscode\github\10.py"
Average Score is 86.25
PS C:\Users\prath\.vscode>
```

Task Description #2 – PEP 8 Compliance

Prompt: generate python code to refactor Python code to follow PEP 8 style guidelines.

def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

correct code:

```
def area_of_rect(length, breadth):
    return length * breadth



print(area_of_rect(10, 20))
```

Explanation:

Added proper spaces and line breaks to make the code readable and follow PEP 8 guidelines. Changed L and B to length and breadth for meaningful variable names.The function multiplies length by breadth to calculate the rectangle's area.

Output:

```
PS C:\Users\prath\.vscode> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
200
PS C:\Users\prath\.vscode>
```

Task Description #3 – Readability Enhancement

Prompt: generate python code to make code more readable without changing its logic.

```
def c(x,y):
return x*y/100
a=200
b=15
print(c(a,b))
```

correct code:

```
# Calculate percentage of a value
def calculate_percentage(value, percentage):
    return value * percentage / 100


total_value = 200
```

percentage_value = 15

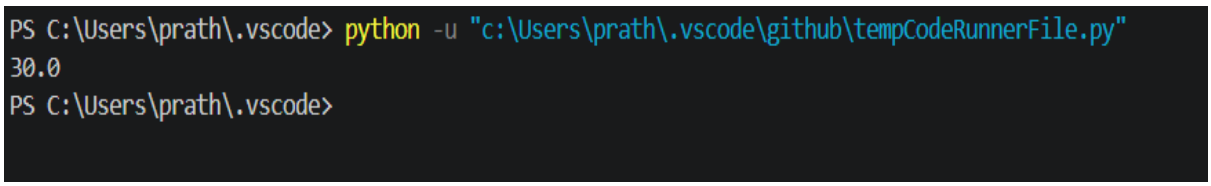
Display the calculated result

print(calculate_percentage(total_value, percentage_value))

Explanation:

Changed variable names (c, x, y, a, b) to descriptive names for better readability.Added comments and proper indentation to make the code easy to understand.The function calculates 15% of 200 and displays the result without changing the original logic.

Output:



Task Description #4 – Refactoring for Maintainability

Prompt: generate python code to break repetitive or long code into reusable functions.

students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])

correct code:

Function to welcome a student

def welcome_student(name):

```
    print("Welcome", name)



students = ["Alice", "Bob", "Charlie"]


# Welcome all students using a loop
for student in students:
    welcome_student(student)
```

Explanation:

Created a reusable function welcome_student() to display a welcome message for each student.Used a for loop to process all students instead of writing repetitive print statements.This makes the code shorter, easier to maintain, and reusable.

Output:

```
PS C:\Users\prath\.vscode> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
30.0
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\prath\.vscode>
```

Task Description #5 – Performance Optimization

Prompt: generate python code to make the code run faster.

```
nums = [i for i in range(1,1000000)]
squares = []

for n in nums:
squares.append(n**2)
print(len(squares))
```

correct code:

```
# Find squares of numbers using list comprehension

squares = [n**2 for n in range(1, 1000000)]


print(len(squares))
```

Explanation:

Used list comprehension to calculate squares in a single line instead of using a separate loop and append(). Removed the unnecessary nums list, reducing extra memory usage. The program calculates the squares of numbers from 1 to 999999 and prints the total count.

Output:

```
PS C:\Users\prath\.vscode> python -u "c:\Users\prath\.vscode\github\tempCodeRunnerFile.py"
999999
PS C:\Users\prath\.vscode>
```

Task Description #6 – Complexity Reduction

Prompt: generate python code to simplify overly complex logic.

```
def grade(score):

if score &gt;= 90:

return &quot;A&quot;

else:

if score &gt;= 80:

return &quot;B&quot;
```

else:

if score >= 70:

return "C"

else:

if score >= 60:

return "D"

else:

return "F"

correct code:

```
def grade(score):
   if score >= 90:
      return "A"
   elif score >= 80:
      return "B"
   elif score >= 70:
      return "C"
   elif score >= 60:
      return "D"
   else:
      return "F"



print(grade(85))
```

Explanation:

Replaced nested if-else statements with elif to simplify the code. The function checks the score and returns the corresponding grade. This makes the code cleaner, easier to read, and easier to maintain.

Output:

